

Tema 09 · AJAX con jQuery y SweetAlert

Clase 023 (23-09) · Teoría: «Servlets» (Clase023_Applet_Servlet.pdf) + guía de estudio (JS, AJAX y SweetAlert)

Programación Avanzada · Prof. Macrino Matías · Universidad del Salvador – Sede Pilar · 2.º cuatrimestre

1. Qué es AJAX

AJAX (Asynchronous JavaScript And XML) es la técnica que permite a una página **comunicarse con el servidor en segundo plano, sin recargarse**. En una aplicación tradicional, cada acción del usuario (enviar un formulario) recarga toda la página. Con AJAX, el JavaScript **envía la petición**, el usuario sigue usando la página y, cuando llega la respuesta, el JS **actualiza solo la parte necesaria** del DOM. Resultado: interfaces más rápidas y fluidas (mejor UX).

A pesar del nombre, hoy el formato de intercambio suele ser **JSON** (o texto) en lugar de XML. «Asincrónico» significa que el navegador **no se bloquea esperando**: registra qué hacer cuando llegue la respuesta (callbacks o promesas).

```
Usuario --- (submit) ---> JavaScript intercepta, evita la recarga (preventDefault)
                        |
                        +--- POST /contexto/registro (en segundo plano) ---> Servlet
                        |
                        |<----- "OK:El cliente X fue registrado" -----+
                        |
                        v
                    JS lee la respuesta y muestra SweetAlert (la página no se recargó)
```

2. Formas de hacer AJAX

Herramienta	Ejemplo	Notas
jQuery \$.ajax (esta clase)	<code>\$.ajax({url, type:'POST', data: \$(form).serialize(), success: fn, error: fn})</code>	Librería que simplifica el DOM y AJAX; se incluye desde un CDN. <code>serialize()</code> arma <code>campo=valor&...</code> con los atributos name
Fetch API (nativa, moderna)	<code>fetch(url, {method:'POST', body: new URLSearchParams(datos)}).then(r => r.text()).then(...)</code>	Sin librerías, basada en promesas. Los datos se envían con URLSearchParams o como JSON (<code>JSON.stringify</code> + cabecera <code>Content-Type: application/json</code>). Es la que menciona la guía de estudio.
XMLHttpRequest	<code>new XMLHttpRequest()</code>	La forma original; hoy casi no se usa directamente

3. El servlet que responde a AJAX

Para el servidor, una petición AJAX es una petición HTTP común: el servlet lee los parámetros con **getParameter** y escribe la respuesta. La diferencia es **qué devuelve**: en vez de HTML completo (que haría hacer forward a una JSP), devuelve solo el dato necesario (un texto o JSON) con el **tipo MIME** correcto (`text/plain` o `application/json`), y no hay forward ni redirect. Detalles correctos: **req.setCharacterEncoding("UTF-8")** antes de leer parámetros (tildes) y **resp.setCharacterEncoding("UTF-8")**.

4. SweetAlert2

SweetAlert2 es una librería que reemplaza los diálogos nativos del navegador (alert, confirm, prompt) por **ventanas modales con mejor diseño**. Se incluye con un CDN (CSS + JS). Se usa con **Swal.fire({ title, text, icon, confirmButtonText })**: el icon puede ser success, error, warning, info, question. Devuelve una promesa: `.then(result => { if (result.isConfirmed) {...} })` para saber si el usuario confirmó. Para confirmar acciones se agrega `showCancelButton: true`. Es uno de los puntos opcionales del parcial (junto con Bootstrap) y la solución lo usa para avisos y confirmaciones.

5. La ruta de contexto (contextPath)

Una aplicación web se publica bajo un **contexto** (el nombre del WAR): `http://localhost:8080/clase-23-09/...` Una URL que empieza con / (por ejemplo '/registro') se interpreta desde la **raíz del servidor, ignorando el contexto**, y devuelve 404. La forma correcta es anteponer el contexto: en JSP con **`${pageContext.request.contextPath}`** (o `request.getContextPath()` en un servlet). Es el error que ilustran los archivos «-err» de la clase.

Seguridad: los datos que llegan por AJAX deben validarse en el servidor igual que los de un formulario; el JS del cliente no es confiable. Y al mostrar texto recibido en el DOM usar `textContent` (no `innerHTML`) para evitar XSS.

Código de las clases, explicado línea por línea

Cada recuadro gris reproduce líneas reales del archivo (con su número de línea) y el recuadro verde de abajo explica qué hacen y por qué están escritas así.

Clase 023 - 23-09: formulario con AJAX (jQuery), servlet de registro y SweetAlert2

Se implementa el registro de un cliente **sin recargar la página**: el formulario (formu-registro.jsp) es interceptado por **cliente.js**, que llama por AJAX al **RegistroServlet**; este responde con un texto OK:... o ERROR:... y el JS lo muestra con **SweetAlert2**. Los archivos «-err» son una versión con errores intencionales (ruta sin contexto) para comparar y aprender a depurarlos.

023 - Clase 23-09 / pom.xml

Qué es y para qué sirve: Archivo de configuración de Maven (POM) de una aplicación web: coordenadas, dependencias y nombre del WAR.

Cómo defenderlo / qué decir: Es el POM que genera el arquetipo de Maven para aplicaciones web. Lo importante: **packaging war**, las APIs de Servlet/JSP como **provided** (las aporta el contenedor) y, si la página usa JSTL, esas librerías sí van en el WAR. Esta versión incluye JSTL.

```
1 | <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2 |   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
```

Líneas 1 a 2: Cabecera del POM: elemento raíz **project** con sus espacios de nombres XML (definen el formato válido del archivo).

```
3 |   <modelVersion>4.0.0</modelVersion>
```

Línea 3: Versión del modelo de POM (siempre 4.0.0).

```
4 |   <groupId>edu.usal</groupId>
5 |   <artifactId>clase-23-09</artifactId>
6 |   <packaging>war</packaging>
7 |   <version>1.0-SNAPSHOT</version>
8 |   <name>clase-23-09 Maven Webapp</name>
9 |   <url>http://maven.apache.org</url>
```

Líneas 4 a 9: Coordenadas del proyecto: **groupId** edu.usal (la organización), **artifactId** clase-23-09 (el nombre), **packaging war** (se empaqueta como aplicación web, un archivo .war que se despliega en Tomcat), versión 1.0-SNAPSHOT, nombre descriptivo y URL (datos del arquetipo maven-archetype-webapp).

```
10 |   <dependencies>
```

Línea 10: Abre la lista de dependencias (librerías que Maven descarga solo).

```
11 |     <dependency>
12 |       <groupId>junit</groupId>
13 |       <artifactId>junit</artifactId>
14 |       <version>3.8.1</version>
15 |       <scope>test</scope>
16 |     </dependency>
17 |
```

Líneas 11 a 17: **JUnit 3.8.1** con **scope test**: solo se usa para pruebas unitarias, no viaja dentro del WAR.

```
18 |     <dependency>
19 |       <groupId>javax.servlet</groupId>
20 |       <artifactId>javax.servlet-api</artifactId>
21 |       <version>4.0.1</version>
22 |       <scope>provided</scope>
23 |     </dependency>
24 |
```

Líneas 18 a 24: **javax.servlet-api 4.0.1** con **scope provided**: se necesita para compilar los servlets, pero en ejecución la provee el servidor (Tomcat 9), por eso no se incluye en el WAR.

```

25 | <dependency>
26 |   <groupId>javax.servlet.jsp</groupId>
27 |   <artifactId>javax.servlet.jsp-api</artifactId>
28 |   <version>2.3.3</version>
29 |   <scope>provided</scope>
30 | </dependency>
31 |

```

Líneas 25 a 31: javax.servlet.jsp-api 2.3.3, también **provided**: la API de JSP que aporta Tomcat.

```

32 | <dependency>
33 |   <groupId>javax.servlet</groupId>
34 |   <artifactId>jstl</artifactId>
35 |   <version>1.2</version>
36 | </dependency>
37 |

```

Líneas 32 a 37: jstl 1.2 (scope compile por defecto): la implementación de JSTL; **Tomcat no la trae**, por eso debe ir dentro del WAR (carpeta WEB-INF/lib).

```

38 | <dependency>
39 |   <groupId>javax.servlet.jsp.jstl</groupId>
40 |   <artifactId>jstl-api</artifactId>
41 |   <version>1.2</version>
42 | </dependency>
43 |

```

Líneas 38 a 43: jstl-api 1.2: la API (interfaces) de JSTL.

```

44 | <dependency>
45 |   <groupId>>taglibs</groupId>
46 |   <artifactId>standard</artifactId>
47 |   <version>1.1.2</version>
48 | </dependency>

```

Líneas 44 a 48: taglibs standard 1.1.2: las librerías de etiquetas estándar (los .tld con core, fmt, etc.).

```

49 | </dependencies>

```

Línea 49: Cierre de la lista de dependencias.

```

50 | <build>
51 |   <finalName>clase-23-09</finalName>
52 | </build>

```

Líneas 50 a 52: finalName define el nombre del WAR generado (clase-23-09.war) y por lo tanto la **ruta de contexto** de la aplicación: http://localhost:8080/clase-23-09/.

```

53 | </project>

```

Línea 53: Cierre del POM.

023 - Clase 23-09 / src / main / webapp / WEB-INF / web.xml

Qué es y para qué sirve: web.xml: descriptor de despliegue de la aplicación web. Vive en WEB-INF y es el archivo de configuración que lee el contenedor (Tomcat) al iniciar.

Cómo defenderlo / qué decir: Aquí solo contiene el nombre de la aplicación porque todavía no hay servlets declarados por XML. Es el lugar donde se declararían servlets, filtros, páginas de error y configuración de sesión (ver Tema 08).

```

1 | <!DOCTYPE web-app PUBLIC
2 |   "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3 |   "http://java.sun.com/dtd/web-app_2_3.dtd" >
4 |

```

Líneas 1 a 4: Declaración del tipo de documento (**DOCTYPE**): indica que es un descriptor de despliegue de aplicación web.

```

5 | <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
6 |         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
7 |         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
8 |                             http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
9 |         version="4.0">

```

Líneas 5 a 9: Elemento raíz **web-app** con los espacios de nombres de Java EE y **version="4.0"** (Servlet 4.0); habilita funciones modernas como las **anotaciones** (@WebServlet) y el EL de JSP 2.3.

```

10 |     <display-name>Archetype Created Web Application</display-name>
11 |
12 |

```

Líneas 10 a 12: Nombre descriptivo de la aplicación (lo muestra el administrador de Tomcat).

```

13 | </web-app>

```

Línea 13: Cierre del descriptor.

023 - Clase 23-09 / src / main / java / edu / usal / servlet / RegistroServlet.java

Qué es y para qué sirve: Servlet que **recibe el registro por POST**, valida que los campos no estén vacíos y responde con un texto «OK:...» o «ERROR:...».

Cómo defenderlo / qué decir: Cumple varios puntos del parcial: **@WebServlet** (anotación), solo **doPost** (modifica datos: método HTTP correcto), **validación en el servidor** (aunque el cliente ya valide) y respuesta de **tipo MIME text/plain** con UTF-8. No hace forward ni redirect porque responde a una llamada AJAX. En una aplicación real aquí se llamaría a un DAO para guardar; la clase solo simula el alta.

```

1 | package edu.usal.servlet;
2 |

```

Líneas 1 a 2: Paquete de los servlets.

```

3 | import javax.servlet.ServletException;
4 | import javax.servlet.annotation.WebServlet;
5 | import javax.servlet.http.HttpServlet;
6 | import javax.servlet.http.HttpServletRequest;
7 | import javax.servlet.http.HttpServletResponse;
8 | import java.io.IOException;
9 | import java.io.PrintWriter;
10 |

```

Líneas 3 a 10: Importaciones: **ServletException**, **WebServlet**, **HttpServlet**, **HttpServletRequest**, **HttpServletResponse**, **IOException**, **PrintWriter**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

11 | @WebServlet("/registro")

```

Línea 11: Anotación: el servlet responde en la URL **/registro** (dentro del contexto de la aplicación). Forma abreviada: solo el patrón.

```

12 | public class RegistroServlet extends HttpServlet {
13 |
14 |     @Override

```

Líneas 12 a 14: Extiende HttpServlet.

```

15 |     protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {

```

Línea 15: Solo se sobreescribe **doPost**: el servlet atiende únicamente peticiones POST (un GET devolvería el error por defecto).

```

16 |         req.setCharacterEncoding("UTF-8");
17 |
18 |         resp.setContentType("text/plain");
19 |         resp.setCharacterEncoding("UTF-8");
20 |

```

Líneas 16 a 20: Fija UTF-8 para **leer** los parámetros (necesario **antes** del primer getParameter) y la respuesta como **text/plain** con UTF-8.

```

21 |     String nombre = req.getParameter("nombre");
22 |     String email = req.getParameter("email");
23 |
24 |     PrintWriter out = resp.getWriter();
25 |

```

Líneas 21 a 25: Lee los parámetros nombre y email (los **name** de los inputs del formulario) y obtiene el **PrintWriter** de la respuesta.

```

26 |         if (nombre != null && !nombre.trim().isEmpty() && email != null && !email.trim().isEmpty()) {
27 |             out.print("OK:El cliente " + nombre + " fue registrado con éxito.");
28 |         } else {
29 |             out.print("ERROR:Todos los campos son obligatorios.");
30 |         }
31 |

```

Líneas 26 a 31: Validación del servidor: ambos deben existir y no estar vacíos (trim()). Si es válido, responde «OK:» más el mensaje de éxito; si no, «ERROR:» más el motivo. Ese prefijo es el «protocolo» que interpreta el JavaScript.

```

32 |         out.flush();
33 |     }
34 | }

```

Líneas 32 a 34: Vacía el buffer para asegurar que la respuesta se envíe.

023 - Clase 23-09 / src / main / webapp / formularios / formu-registro.jsp

Qué es y para qué sirve: Página (JSP) con el **formulario de registro de clientes** que se envía con AJAX.

Cómo defenderlo / qué decir: Es una JSP porque necesita que el servidor calcule el **contextPath** (con EL) y se lo pase al JavaScript; con HTML estático eso no sería posible. El formulario no tiene action: el **submit** lo captura jQuery.

```

1 | <%--
2 |   Created by IntelliJ IDEA.
3 |   User: usal
4 |   Date: 23/9/26
5 |   Time: 1:23 p. m.
6 |   To change this template use File | Settings | File Templates.
7 | --%>

```

Líneas 1 a 7: Comentario JSP generado por el IDE; no llega al navegador.

```

8 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>

```

Línea 8: Directiva page: tipo de contenido y codificación UTF-8.

```

9 | <html>
10 | <head>
11 |     <title>Clase 23-09 | Demo Ajax</title>

```

Líneas 9 a 11: Apertura de html y head con el título «Clase 23-09 | Demo Ajax».

```

12 |     <!-- Bootstrap 5 -->
13 |     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">

```

Líneas 12 a 13: Hoja de estilos de **Bootstrap 5.3.3** desde un CDN.

```

14 |     <!-- SweetAlert2 CSS -->
15 |     <link href="https://cdn.jsdelivr.net/npm/sweetalert2@11/dist/sweetalert2.min.css" rel="stylesheet">

```

Líneas 14 a 15: Hoja de estilos de **SweetAlert2**: da formato a las ventanas modales.

```

16 |     <!-- 1. Librerías Externas (CDN) AJAX -->
17 |     <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
18 |     <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
19 | </head>

```

Líneas 16 a 19: **Librerías JavaScript externas (CDN): jQuery 3.7.1** (para \$ y \$.ajax) y **SweetAlert2** (Swal.fire). Se cargan en el head, antes del código propio que las usa.

```

20 | <body class="bg-light p-5">
21 |
22 | <div class="container" style="max-width: 450px;">
23 |   <div class="card shadow">
24 |     <div class="card-header bg-primary text-white">
25 |       <h5 class="mb-0">Registrar Cliente</h5>
26 |     </div>
27 |     <div class="card-body">
28 |

```

Líneas 20 a 28: Body con fondo claro y padding. container de ancho máximo 450px centrado y una **tarjeta** (card) con encabezado azul «Registrar Cliente».

```

29 |         <!-- Formulario -->
30 |         <form id="formCliente">
31 |           <div class="mb-3">
32 |             <label for="nombre" class="form-label">Nombre Completo:</label>
33 |             <input type="text" id="nombre" name="nombre" class="form-control" >
34 |           </div>
35 |
36 |           <div class="mb-3">
37 |             <label for="email" class="form-label">Correo Electrónico:</label>
38 |             <input type="email" id="email" name="email" class="form-control" >
39 |           </div>
40 |

```

Líneas 29 a 40: Formulario con id="formCliente" (el id que busca el JS). **No tiene action ni method:** no se envía de forma tradicional; lo intercepta JavaScript. Dos campos: nombre (text) y email (email); los **name** son los parámetros que leerá el servlet.

```

41 |             <button type="submit" class="btn btn-success w-100">Guardar Cliente</button>
42 |           </form>
43 |
44 |         </div>
45 |       </div>
46 |     </div>
47 |

```

Líneas 41 a 47: Botón **submit** «Guardar Cliente»: dispara el evento submit que captura el JS.

```

48 |     <script>
49 |       const contextPath = '${pageContext.request.contextPath}';
50 |     </script>

```

Líneas 48 a 50: Script en línea que define la constante **contextPath** con **EL**: '\${pageContext.request.contextPath}'. Como la JSP se procesa en el servidor, el navegador recibe el valor real (por ejemplo '/clase-23-09'). Es la técnica para que el JS externo conozca el contexto.

```

51 |     <script src="../../js/cliente.js"></script>
52 | </body>
53 | </html>

```

Líneas 51 a 53: Carga el JavaScript de la lógica AJAX (**cliente.js**), después de definir contextPath.

023 - Clase 23-09 / src / main / webapp / js / cliente.js

Qué es y para qué sirve: Código JavaScript con **jQuery** que intercepta el envío del formulario, hace una **petición AJAX (POST)** al servlet y muestra la respuesta con **SweetAlert2**.

Cómo defenderlo / qué decir: **AJAX** permite comunicarse con el servidor **sin recargar la página:** el JS envía los datos en segundo plano y, cuando llega la respuesta, actualiza solo lo necesario. El protocolo acordado con el servlet es simple: la respuesta es texto con un **prefijo** («OK:» o «ERROR:») seguido del mensaje; el JS lo interpreta. En proyectos reales se suele usar **JSON**. La guía de estudio menciona también la **Fetch API**, la forma moderna sin jQuery.

```

1 | $(document).ready(function() {
2 |

```

Líneas 1 a 2: \$(document).ready(...): jQuery ejecuta esta función cuando el HTML terminó de cargar (el DOM está listo), igual que esperar el evento load.

```

3 |     $('#formCliente').on('submit', function(e) {
4 |
5 |         e.preventDefault();
6 |
7 |

```

Líneas 3 a 7: Listener del evento submit: \$('#formCliente') selecciona el formulario por su id (sintaxis CSS: # = id). **e.preventDefault()** cancela el envío normal del formulario (que recargaría la página).

```

8 |         $.ajax({

```

Línea 8: \$.ajax({...}): abre la configuración de la petición AJAX.

```

9 |             url: contextPath + '/registro',

```

Línea 9: url: contextPath + '/registro': la URL del servlet. **contextPath** es una constante que la JSP define con EL ('\${pageContext.request.contextPath}'), por lo que se obtiene /clase-23-09/registro sin importar con qué nombre se despliegue el WAR.

```

10 |             type: 'POST',
11 |             data: $(this).serialize(), // Empaqueta 'nombre' y 'email'
12 |             dataType: 'text',          // Esperamos respuesta de tipo TEXTO

```

Líneas 10 a 12: type: 'POST': método HTTP (se modifica/crea datos). **data: \$(this).serialize()** convierte todos los campos del formulario en el texto nombre=...&email=... (usa los atributos **name** de los inputs). **dataType: 'text':** se espera una respuesta de texto.

```

13 |             success: function(respuestaTexto) {
14 |
15 |                 // 3. Evaluamos la respuesta de texto recibida desde el Servlet

```

Líneas 13 a 15: success: se ejecuta si el servidor respondió bien (código 2xx). Recibe el texto de la respuesta.

```

16 |                 if (respuestaTexto.startsWith("OK:")) {
17 |
18 |                     // Extraemos el mensaje removiendo el prefijo "OK:"
19 |                     let mensajeExito = respuestaTexto.replace("OK:", "");
20 |
21 |                     // Mostramos el diálogo emergente de SweetAlert2

```

Líneas 16 a 21: Evalúa la respuesta: si **empieza con «OK:»** la operación fue exitosa, y `replace("OK:", "")` deja solo el mensaje.

```

22 |                     Swal.fire({
23 |                         title: '¡Éxito!',
24 |                         text: mensajeExito,
25 |                         icon: 'success',
26 |                         confirmButtonText: 'Aceptar'

```

Líneas 22 a 26: Swal.fire({...}) de SweetAlert2: muestra una ventana modal bonita (en lugar del feo alert del navegador) con título «¡Éxito!», texto, ícono success y botón «Aceptar».

```

27 |                     }).then((result) => {
28 |                         // Al presionar "Aceptar", limpiamos el formulario
29 |                         if (result.isConfirmed) {
30 |                             $('#formCliente')[0].reset();
31 |                         }
32 |                     });
33 |

```

Líneas 27 a 33: .then(...): Swal devuelve una **promesa**; el código se ejecuta cuando el usuario cierra la ventana. Si **result.isConfirmed** (presionó Aceptar) limpia el formulario con **reset()** (el [0] obtiene el elemento DOM puro del objeto jQuery).

```

34 |         } else {
35 |
36 |             // Extraemos el mensaje de error removiendo el prefijo "ERROR:"
37 |             let mensajeError = respuestaTexto.replace("ERROR:", "");
38 |
39 |             Swal.fire({
40 |                 title: 'Atención',
41 |                 text: mensajeError,
42 |                 icon: 'warning'
43 |             });
44 |
45 |         },
46 |     },

```

Líneas 34 a 46: Si la respuesta **no** empieza con «OK:» (es «ERROR:...»), quita el prefijo y muestra el mensaje con ícono de **advertencia** (warning).

```

47 |         error: function() {
48 |             Swal.fire({
49 |                 title: 'Error',
50 |                 text: 'No se pudo comunicar con el servidor.',
51 |                 icon: 'error'
52 |             });
53 |         },
54 |     });
55 | });
56 |
57 | });

```

Líneas 47 a 57: error: se ejecuta si la **comunicación falla** (servidor caído, 404, 500, red). Muestra un SweetAlert con ícono de error.

023 - Clase 23-09 / src / main / webapp / formularios / formu-registro-err.jsp

Qué es y para qué sirve: Página (JSP) con el **formulario de registro de clientes** que se envía con AJAX. Es la versión **con error** de la demostración.

Cómo defenderlo / qué decir: Versión pensada para mostrar los **errores típicos**: el JS usa una URL sin contexto (por eso falla) y se agrega un botón con alert() clásico para comparar con SweetAlert. Es una JSP porque necesita que el servidor calcule el **contextPath** (con EL) y se lo pase al JavaScript; con HTML estático eso no sería posible. El formulario no tiene action: el **submit** lo captura jQuery.

```

1 | <%--
2 |   Created by IntelliJ IDEA.
3 |   User: usal
4 |   Date: 23/9/26
5 |   Time: 1:23 p. m.
6 |   To change this template use File | Settings | File Templates.
7 | --%>

```

Líneas 1 a 7: Comentario JSP generado por el IDE; no llega al navegador.

```

8 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>

```

Línea 8: Directiva page: tipo de contenido y codificación UTF-8.

```

9 | <html>
10 | <head>
11 |   <title>Clase 23-09 | Demo Ajax</title>

```

Líneas 9 a 11: Apertura de html y head con el título «Clase 23-09 | Demo Ajax».

```

12 | <!-- Bootstrap 5 -->
13 | <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">

```

Líneas 12 a 13: Hoja de estilos de **Bootstrap 5.3.3** desde un CDN.

```

14 | <!-- SweetAlert2 CSS -->
15 | <link href="https://cdn.jsdelivr.net/npm/sweetalert2@11/dist/sweetalert2.min.css" rel="stylesheet">

```

Líneas 14 a 15: Hoja de estilos de **SweetAlert2**: da formato a las ventanas modales.

```

16 | <!-- Librerías Externas (CDN) AJAX -->
17 | <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
18 | <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
19 | </head>

```

Líneas 16 a 19: Librerías JavaScript externas (CDN): jQuery 3.7.1 (para \$ y \$.ajax) y **SweetAlert2** (Swal.fire). Se cargan en el head, antes del código propio que las usa.

```

20 | <body class="bg-light p-5">
21 |
22 | <div class="container" style="max-width: 450px;">
23 |   <div class="card shadow">
24 |     <div class="card-header bg-primary text-white">
25 |       <h5 class="mb-0">Registrar Cliente</h5>
26 |     </div>
27 |     <div class="card-body">
28 |

```

Líneas 20 a 28: Body con fondo claro y padding. container de ancho máximo 450px centrado y una **tarjeta** (card) con encabezado azul «Registrar Cliente».

```

29 |       <!-- Formulario -->
30 |       <form id="formCliente">
31 |         <div class="mb-3">
32 |           <label for="nombre" class="form-label">Nombre Completo:</label>
33 |           <input type="text" id="nombre" name="nombre" class="form-control" >
34 |         </div>
35 |
36 |         <div class="mb-3">
37 |           <label for="email" class="form-label">Correo Electrónico:</label>
38 |           <input type="email" id="email" name="email" class="form-control" >
39 |         </div>
40 |

```

Líneas 29 a 40: Formulario con id="formCliente" (el id que busca el JS). **No tiene action ni method**: no se envía de forma tradicional; lo intercepta JavaScript. Dos campos: nombre (text) y email (email); los **name** son los parámetros que leerá el servlet.

```

41 |       <button type="submit" class="btn btn-success w-100">Guardar Cliente</button>

```

Línea 41: Botón submit «Guardar Cliente»: dispara el evento submit que captura el JS.

```

42 |       <button type="button" class="my-3 btn btn-secondary w-100" onclick="alert('Mensaje sin
43 |         ↳ SweetAlert')">Mensaje Clasico</button>
44 |     </form>
45 |   </div>
46 | </div>
47 | </div>
48 |

```

Líneas 42 a 48: Botón type="button" (no envía el formulario) con **onclick="alert(...)"**: muestra el **mensaje clásico** del navegador, para comparar con SweetAlert.

```

49 | <script src="../js/cliente-err.js"></script>
50 | </body>
51 | </html>

```

Líneas 49 a 51: Carga el JS de la versión con error. La ruta ../js/ es relativa a la carpeta formularios. Esta versión **no define contextPath**.

023 - Clase 23-09 / src / main / webapp / js / cliente-err.js

Qué es y para qué sirve: Código JavaScript con **jQuery** que intercepta el envío del formulario, hace una **petición AJAX (POST)** al servlet y muestra la respuesta con **SweetAlert2**. Es la versión **con error** (ver defensa).

Cómo defenderlo / qué decir: Se conserva como **ejemplo del error clásico**: usar una ruta absoluta que ignora el contexto de la aplicación. La diferencia con cliente.js es una sola línea (la url) y otra en la JSP (que acá no define contextPath). **AJAX** permite comunicarse con el servidor **sin recargar la página**: el JS envía los datos en segundo plano y, cuando llega la respuesta, actualiza solo lo necesario. El protocolo acordado con el servlet es simple: la respuesta es texto con un **prefijo** («OK:» o «ERROR:») seguido del mensaje; el JS lo interpreta. En proyectos reales se suele usar **JSON**. La guía de estudio menciona también la **Fetch API**, la forma moderna sin jQuery.

```

1 | $(document).ready(function() {
2 |

```

Líneas 1 a 2: `$(document).ready(...)`: jQuery ejecuta esta función cuando el HTML terminó de cargar (el DOM está listo), igual que esperar el evento load.

```

3 |     $('#formCliente').on('submit', function(e) {
4 |
5 |         e.preventDefault();
6 |
7 |

```

Líneas 3 a 7: Listener del evento submit: `$('#formCliente')` selecciona el formulario por su id (sintaxis CSS: `# = id`). **`e.preventDefault()`** cancela el envío normal del formulario (que recargaría la página).

```

8 |         $.ajax({

```

Línea 8: `$.ajax({...})`: abre la configuración de la petición AJAX.

```

9 |             url: '/registro',

```

Línea 9: url: `'/registro'`: **ERROR** de esta versión: la ruta **empieza en la raíz del servidor** (`http://localhost:8080/registro`) y no incluye el **contexto** de la aplicación (`/clase-23-09`). El servidor responde 404, jQuery ejecuta la función error y el usuario ve «No se pudo comunicar con el servidor».

```

10 |             type: 'POST',
11 |             data: $(this).serialize(), // Empaqueta 'nombre' y 'email'
12 |             dataType: 'text',          // Esperamos respuesta de tipo TEXTO

```

Líneas 10 a 12: type: `'POST'`: método HTTP (se modifica/crea datos). **data: `$(this).serialize()`** convierte todos los campos del formulario en el texto `nombre=...&email=...` (usa los atributos **name** de los inputs). **dataType: `'text'`:** se espera una respuesta de texto.

```

13 |             success: function(respuestaTexto) {
14 |
15 |                 // 3. Evaluamos la respuesta de texto recibida desde el Servlet

```

Líneas 13 a 15: success: se ejecuta si el servidor respondió bien (código 2xx). Recibe el texto de la respuesta.

```

16 |                 if (respuestaTexto.startsWith("OK:")) {
17 |
18 |                     // Extraemos el mensaje removiendo el prefijo "OK:"
19 |                     let mensajeExito = respuestaTexto.replace("OK:", "");
20 |
21 |                     // Mostramos el diálogo emergente de SweetAlert2

```

Líneas 16 a 21: Evalúa la respuesta: si **empieza con «OK:»** la operación fue exitosa, y `replace("OK:", "")` deja solo el mensaje.

```

22 |                     Swal.fire({
23 |                         title: '¡Éxito!',
24 |                         text: mensajeExito,
25 |                         icon: 'success',
26 |                         confirmButtonText: 'Aceptar'

```

Líneas 22 a 26: `Swal.fire({...})` de **SweetAlert2:** muestra una ventana modal bonita (en lugar del feo alert del navegador) con título «¡Éxito!», texto, ícono success y botón «Aceptar».

```

27 |                     }).then((result) => {
28 |                         // Al presionar "Aceptar", limpiamos el formulario
29 |                         if (result.isConfirmed) {
30 |                             $('#formCliente')[0].reset();
31 |                         }
32 |                     });
33 |

```

Líneas 27 a 33: `.then(...)`: Swal devuelve una **promesa**; el código se ejecuta cuando el usuario cierra la ventana. Si **result.isConfirmed** (presionó Aceptar) limpia el formulario con **reset()** (el `[0]` obtiene el elemento DOM puro del objeto jQuery).

```

34 |         } else {
35 |
36 |             // Extraemos el mensaje de error removiendo el prefijo "ERROR:"
37 |             let mensajeError = respuestaTexto.replace("ERROR:", "");
38 |
39 |             Swal.fire({
40 |                 title: 'Atención',
41 |                 text: mensajeError,
42 |                 icon: 'warning'
43 |             });
44 |
45 |         },
46 |     },

```

Líneas 34 a 46: Si la respuesta **no** empieza con «OK:» (es «ERROR:...»), quita el prefijo y muestra el mensaje con ícono de **advertencia** (warning).

```

47 |         error: function() {
48 |             Swal.fire({
49 |                 title: 'Error',
50 |                 text: 'No se pudo comunicar con el servidor.',
51 |                 icon: 'error'
52 |             });
53 |         }
54 |     });
55 | });
56 |
57 | });

```

Líneas 47 a 57: error: se ejecuta si la **comunicación falla** (servidor caído, 404, 500, red). Muestra un SweetAlert con ícono de error.

Resumen para estudiar

- **AJAX** = petición asincrónica al servidor desde JavaScript, sin recargar la página; el JS actualiza solo el DOM.
- jQuery: `$(document).ready, $('#id').on('submit', ...), e.preventDefault(), $.ajax({url, type, data, success, error}), serialize()`. Alternativa moderna: **fetch**.
- El servlet de AJAX responde **solo el dato** (texto/JSON) con el tipo MIME correcto; no usa forward a una JSP.
- **SweetAlert2**: `Swal.fire({title, text, icon})` → promesa `.then(result.isConfirmed)`.
- **contextPath**: nunca usar rutas que empiecen con / sin el contexto: en JSP `${pageContext.request.contextPath}`.
- Validar siempre también en el servidor.

Cómo se aplica en el parcial de logística

- **app.js** de la solución usa SweetAlert para mostrar los avisos del servidor y para confirmar altas, bajas e inicio/fin de viajes. La guía de estudio propone además enviar el origen y el destino del viaje con **fetch**; la solución entregada resuelve el flujo con formularios y GET/POST tradicionales (más simple y robusto), pudiéndose migrar a fetch con el mismo servidor.

Preguntas típicas

¿Qué ventaja tiene AJAX?

No recarga toda la página: es más rápido y mejora la experiencia del usuario, porque solo se intercambia y actualiza lo necesario.

¿Por qué falla la versión «-err»?

Porque usa la URL '/registro' sin el contexto de la aplicación: apunta a la raíz del servidor, donde no existe el servlet, y devuelve 404. Hay que usar `contextPath + '/registro'`.

¿Para qué sirve `e.preventDefault()`?

Cancela el envío normal del formulario (que recargaría la página) para que sea el JavaScript quien haga la petición AJAX.